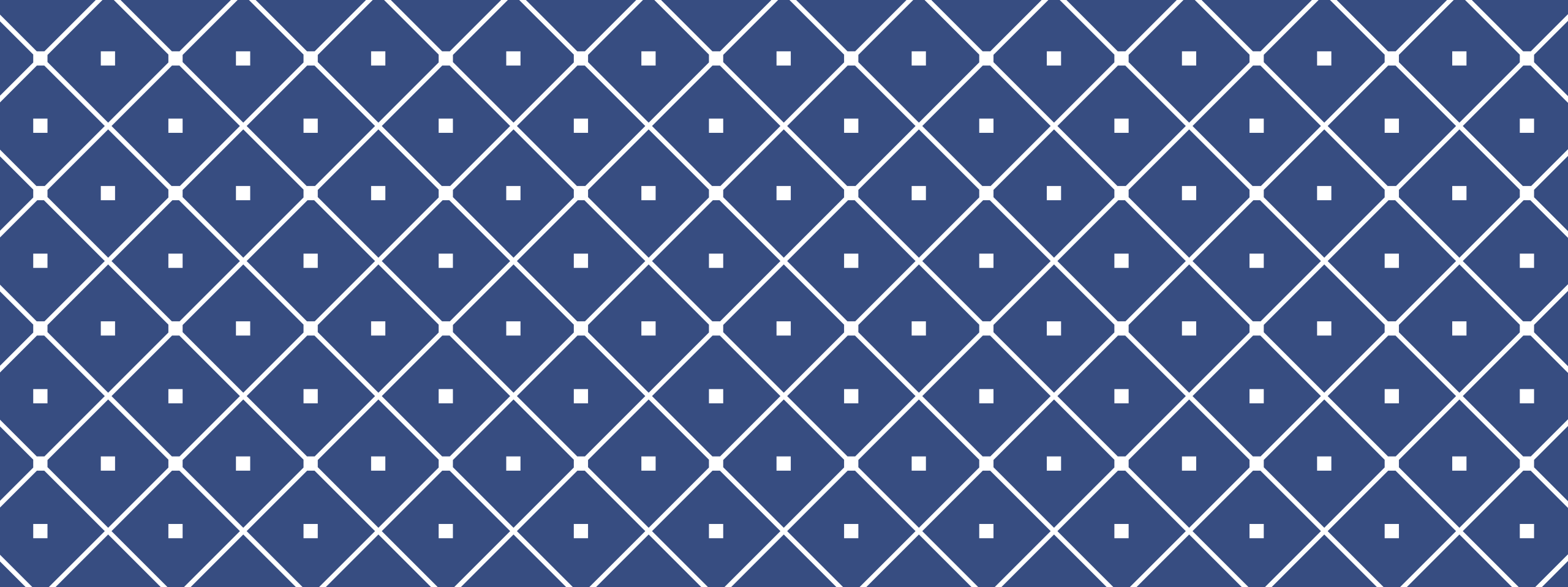


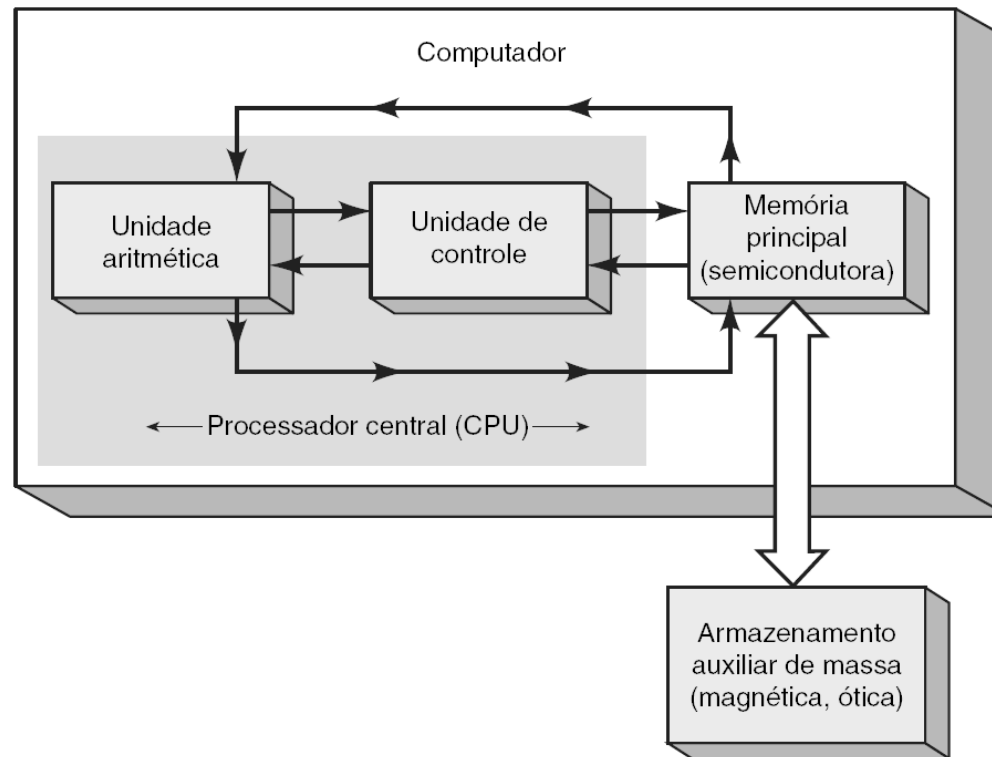


HIERARQUIA DE MEMÓRIA

Fundamentos de Arquitetura de
Computadores

INTRODUÇÃO

Memórias semicondutoras são usadas como memória principal de um computador quando a velocidade de operação é importante



MEMÓRIA IDEAL

Tempo de acesso (latência) zero 

- Tempo decorrido entre a requisição de um item de dado pelo processador e o efetivo recebimento deste item

Capacidade infinita 

Custo zero 

Largura de banda infinita (para suportar múltiplos acessos em paralelo) 

MEMÓRIA IDEAL

Problemas

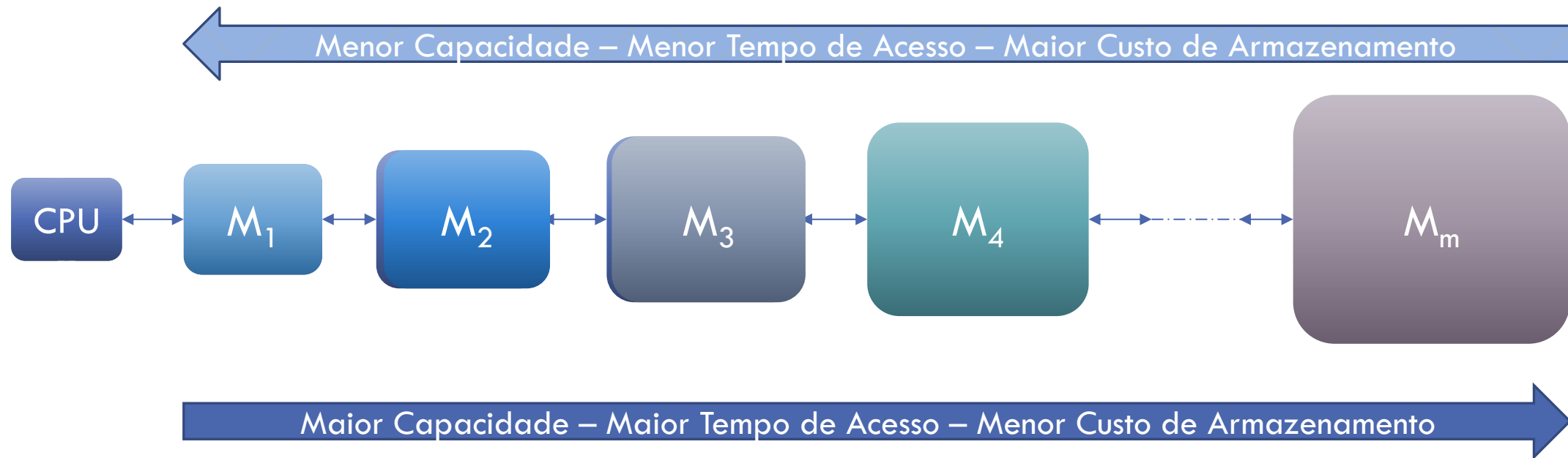
- Quanto maior → mais lenta
- Quanto mais rápido → mais caro
- Largura de Banda é mais caro
 - Tecnologia mais rápida
 - Maiores frequências
 - Mais portas
 - Mais bancos de memórias

Tipo	Tempo de Acesso	Valor por GB @ 2015
SRAM	0,5 ns	US\$ 5.000
DRAM	10 a 50 ns	US\$ 7
SSD	20.000 ns	US\$ 0,40
HDD	5.000.000 ns	US\$ 0,05

MEMÓRIA IDEAL

Solução

- Prover múltiplos níveis de armazenagem (progressivamente maiores e mais lentos ao se afastar do processador) e garantir que a maioria dos dados necessários pelo processador sejam mantidos nos níveis mais rápidos



HIERARQUIA DE MEMÓRIA

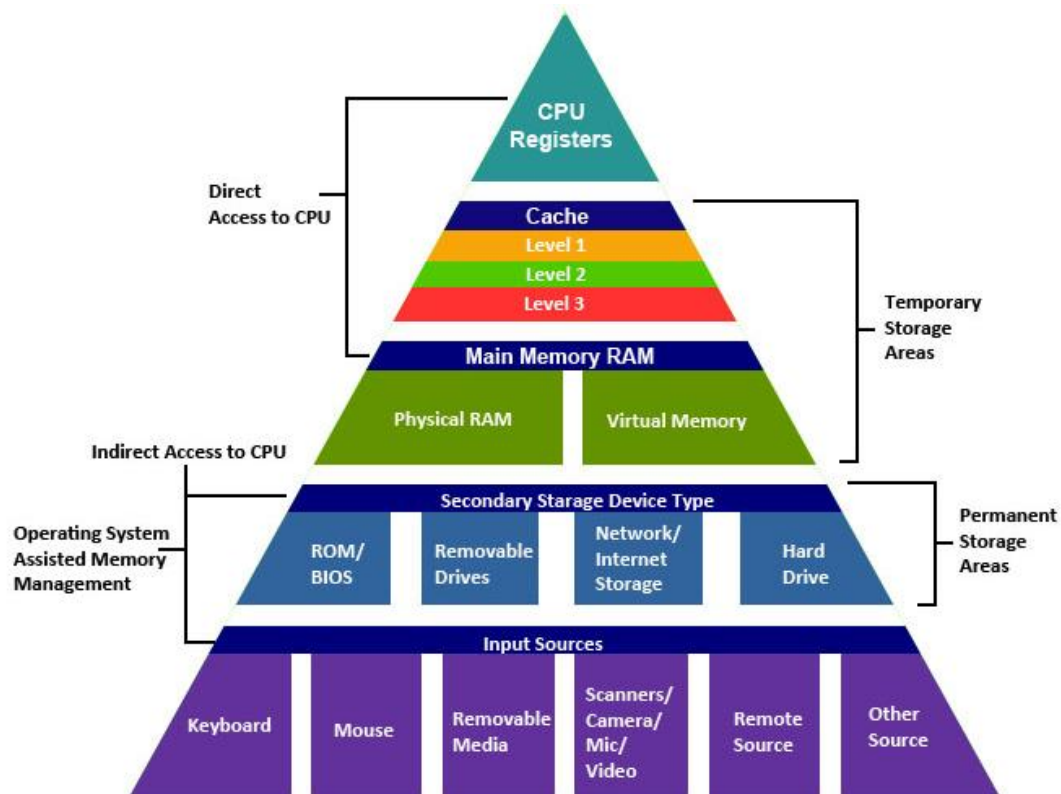
Objetivo

- Estabelecer um sistema de memória que aparenta ao usuário ter grande capacidade de armazenamento e alta velocidade de acesso a um custo razoável

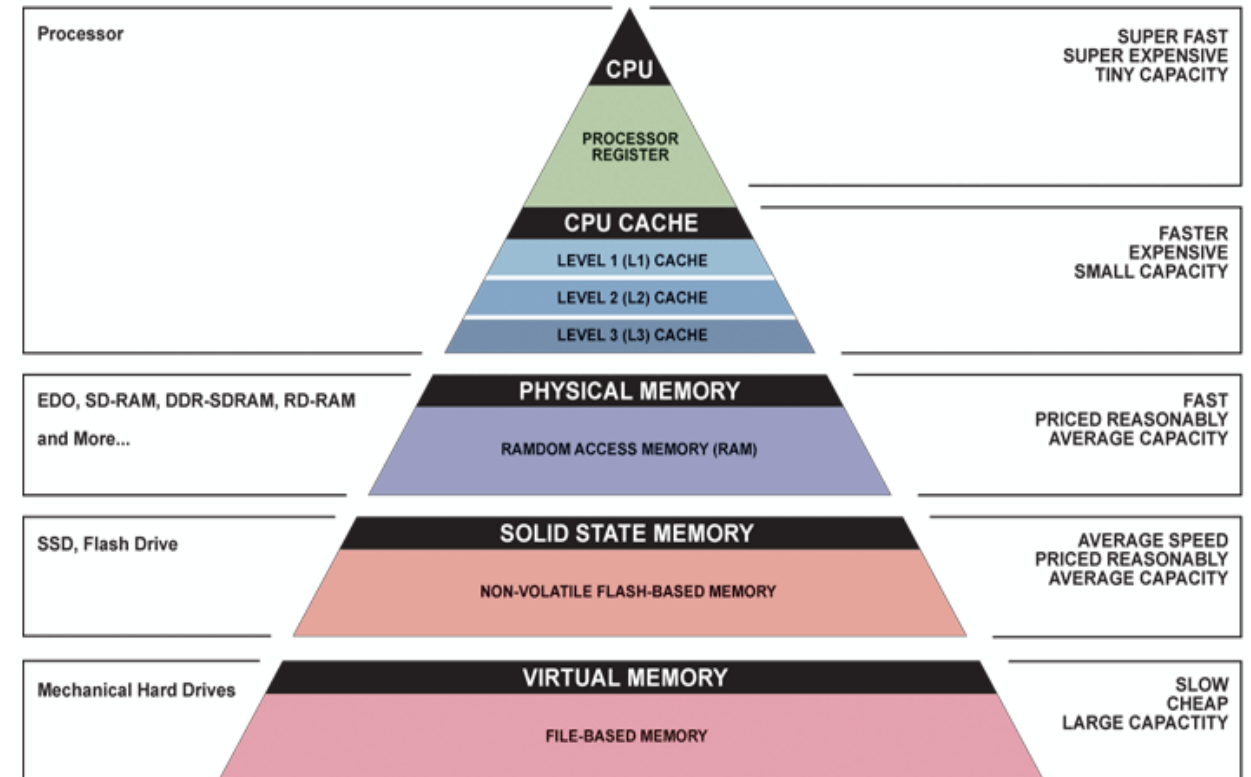
Metodologia

- Estruturação em níveis de memória $M_1, M_2, \dots, M_i$ tal que M_i é mais rápida porém menor que M_{i+1}

HIERARQUIA DE MEMÓRIA



<http://lib.bioinfo.pl/courses/view/543>



<https://sites.google.com/site/cachememory2011/memory-hierarchy>

HIERARQUIA DE MEMÓRIA: PROPRIEDADES

Inclusão

- Cada conjunto de dados M_i deve estar contido no conjunto de dados em M_{i+1}
$$M_1 \subset M_2 \subset \dots \subset M_n$$

Coerência

- Cópias de um mesmo item devem ser consistentes ao longo de níveis sucessivos da hierarquia de memória

Localidade de Referência

- Referências à memória geradas pela CPU acessam endereços que estejam agrupados em certas regiões (ou sequências) no tempo ou no espaço

ARMAZENAMENTO

O armazenamento de bytes de um elemento na memória pode ser diferente entre arquiteturas de computador

- Todas são endereçáveis por byte
- Diferenças em elementos básicos (int, float, ...) que possuem mais de 1 byte

A diferença está na sequência dos bytes armazenados

- Pode causar problemas quando dados são transferidos de uma arquitetura para a outra

Tipos

- *Little Endian*
- *Big Endian*

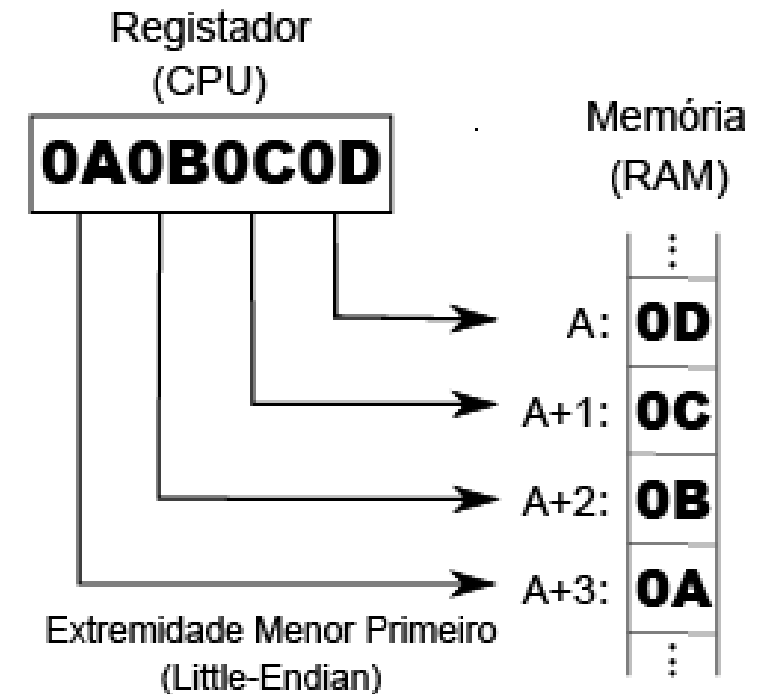
ARMAZENAMENTO: LITTLE ENDIAN

Armazena a partir do byte menos significativo para o mais significativo (LSB → MSB)

Vantagens

- Facilita a adição e multiplicação de números de alta precisão
- Permite leitura/escrita em endereço ímpares

Utilizado pelas arquiteturas x86 (Intel, AMD,...)



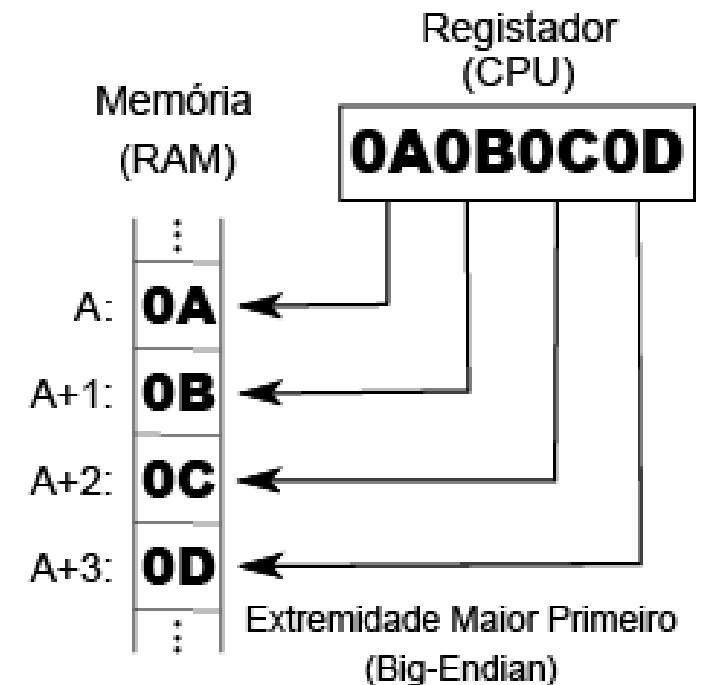
ARMAZENAMENTO: BIG ENDIAN

Armazena a partir do byte mais significativo para o menos significativo (MSB → LSB)

Vantagens

- Facilidade em determinar o sinal de um número (no *little endian*, tem que saber quantos bytes ocupa para encontrar o bit de sinal)
- Operações com strings são mais eficientes
- Boa parte das representações gráficas são big endian (Adobe Photoshop, JPEG, MacPaint, Arquivos Tracer Sun)

Utilizado pelas arquiteturas ARM (RISC)



ARMAZENAMENTO: BI-ENDIAN

Permite o armazenamento nos dois formatos

- Arquiteturas ARM, PowerPC, DEC Alpha, SPARC V9, MIPS, PA-RISC, SuperH SH-4 e IA-64

Estabelecido pelo sistema operacional em que modo o processo deve ser executado

Formatos Microsoft WAV/AVI, arquivos TIF e XWD permitem codificar ambos os tipos